

AI Smart

AI 小智 小型嵌入式智能助手，使用deepseek

1. 项目基本信息

名称简介：

名称：AI小智

简介：小型嵌入式智能助手设备，接入目前主流的Deepseek大模型，配备350*350分辨率屏幕，时间日期显示，触摸屏，对话面板，通过文本进行交互（后续支持语音），支持日常问题沟通交流

项目状态

- 开发中

待改进

开发板网络部署-SSL证书

2. 功能特性

核心功能

deepseek

响应解析：解析json响应 并提取内容

对话管理：创建销毁会话，维护api key，

文本分析：语气分析，情绪回应

令牌统计：跟踪API使用情况和成本估算

用户界面：

多界面交互

动态表情：随即切换的动画表情

时间显示：实时时钟和日期

手势切换：手势滑动切换屏幕

输入系统：

虚拟键盘：集成LVGL键盘组件

文本输入：支持多行文本输入

语音交互：语音输入支持（待实现）

主要特点

可以对文本进行分析并回应，进行语气调整和情绪回应等

知识系统：

通用知识解答

实时信息查询

学习能力：

辅助功能：

工具集成：

时间管理、日历天气

计算功能、翻译功能

项目截图/演示

界面

数量：3

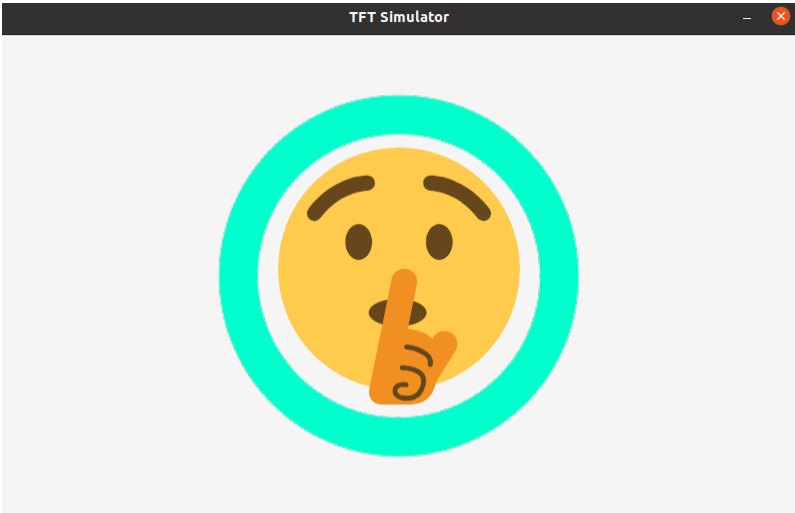
时间界面：

表情图：10个，间隔1秒内随机切换，循环

切换方式：新旧交替闪现

实现：定时器

进入：通过主界面向下滑动进入



切换方式：点击

1.主界面：

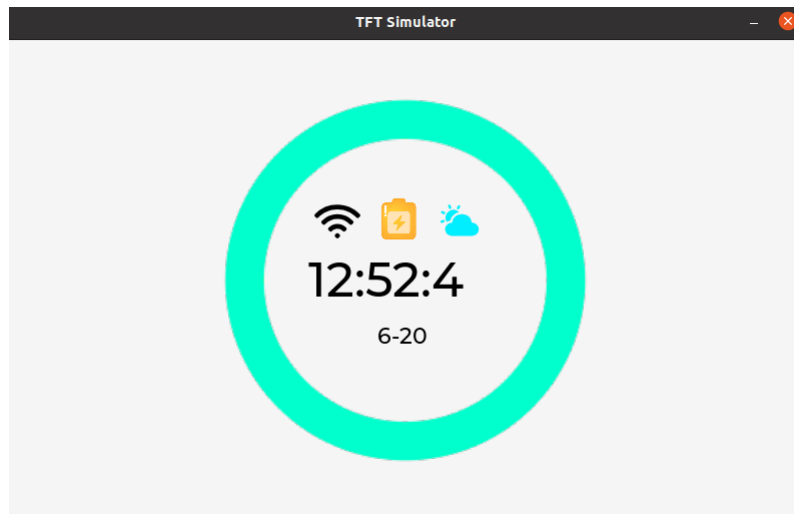
内容：

电池logo

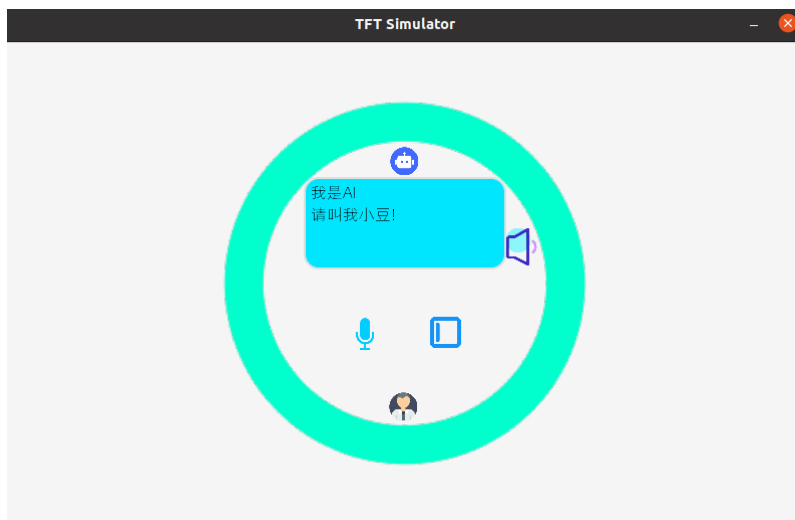
WiFi logo

日历

时钟：时分秒



2.AI对话：使用虚拟键盘，点击键盘输入，暂时只支持 英文



键盘输入面板：



3.核心功能实现

常态界面

在屏幕加载事件LV_EVENT_LOADED中创建定时器，并设置定时器回调函数，回调函数对表情索引进行随机取值，取一个与之前不同的索引进行展示，定时器间隔一秒调用。

```
/******  
 * @brief 初始化随机数生成器，并创建定时器  
 * @param e  
 * @details 创建定时器，每一秒执行一次回调函数  
 *****/  
void InitEmojiAutoChange(lv_event_t *e)  
{  
    lv_event_code_t code = lv_event_get_code(e);  
    lv_obj_t * ta = lv_event_get_target(e);  
    if(code != LV_EVENT_SCREEN_LOADED) return; // 确保只在屏幕加载时执行一次  
    if( code == LV_EVENT_SCREEN_LOADED )  
    {  
        lv_obj_t *screen = lv_event_get_target(e);  
        srand(time(NULL));  
        emoji_timer = lv_timer_create(emoji_change_timer_cb, 1000, screen);  
        printf("创建定时器! \n");  
    }  
}
```

```
/******  
 * @brief emoji表情切换定时器回调函数  
 * @param timer 定时器  
 * @param idx 显示表情索引  
 * @param last_idx 上一个显示索引  
 * @details 获取定时器，获取父控件，获取父控件下子控件的个数  
 *           生成一个范围在子控件个数内的随机索引，作为显示表情的索引，显示当前索引表情并隐藏上一个  
 *           索引表情  
 *****/  
static void emoji_change_timer_cb(lv_timer_t *timer)  
{  
    lv_obj_t *screen = (lv_obj_t *)timer->user_data;  
    lv_obj_t * container = lv_obj_get_child(screen, 1);  
    uint16_t child_cnt = lv_obj_get_child_cnt(container);  
  
    if(child_cnt == 0) return;  
    // 生成一个与上次不同的随机索引  
    uint16_t idx;  
    do {  
        idx = rand() % child_cnt;  
        printf("生成随机索引!: %d\n", idx);  
    } while(idx == last_idx && child_cnt > 1);  
  
    // 隐藏表情  
    printf("last_idx 索引为 %d\n", last_idx);  
    lv_obj_add_flag(lv_obj_get_child(container, last_idx), LV_OBJ_FLAG_HIDDEN);
```

```

// 显示选中的表情
lv_obj_t *curImage = lv_obj_get_child(container, idx);
lv_obj_clear_flag(curImage, LV_OBJ_FLAG_HIDDEN);
printf("显示表情! %d\n", idx);
// 随机设置下一次切换的时间间隔（1-3秒）
lv_timer_set_period(timer, 1000 + rand() % 2000);

last_idx = idx;
}

```

时间界面

在屏幕加载事件LV_EVENT_LOADED中也同步创建定时器，并设置定时器回调函数，该定时器设置在常态界面的加载事件中以提前消除加载该界面时的短暂空白延时，时间数据使用time结构体获取不同的时间成员变量（显示label的长度要稍微增加一点，例如加5）。

```

/*****
 * @brief 时间日期显示定时器初始化
 * @param e
 * @param timeDateTimer 全局变量，时间日期定时器
 *****/
void InitSowTimeDate(lv_event_t *e)
{
    lv_event_code_t code = lv_event_get_code(e);
    lv_obj_t * ta = lv_event_get_target(e);
    if(code != LV_EVENT_SCREEN_LOADED) return; // 确保只在屏幕加载时执行一次
    if( code == LV_EVENT_SCREEN_LOADED )
    {
        timeDateTimer = lv_timer_create(TimeDateTimercb, 1000, NULL);
    }
}

```

```

/*****
 * @brief 获取时间
 *****/
void GetOutNowTime()
{
    time_t now;
    struct tm *local_time;
    time(&now);
    local_time = localtime(&now);

    int month = local_time->tm_mon + 1;
    // 月份从0开始，所以加1
    int day = local_time->tm_mday;
    int hour = local_time->tm_hour;
    int min = local_time->tm_min;
    int sec = local_time->tm_sec;
    // printf("今天是 %d月%d日%d时%d分%d秒\n", month, day, hour, min, sec);
}

```

```

        lv_label_set_text_fmt(ui_time2, "%d:%d:%d", hour, min, sec);

    }

```

```

/*****
 * @brief 获取日期
 *****/
void GetOutNowDate()
{
    time_t now;
    struct tm *local_time;
    time(&now);
    local_time = localtime(&now);

    int year = local_time->tm_year;
    int month = local_time->tm_mon + 1;
    // 月份从0开始，所以加1
    int day = local_time->tm_mday;
    int hour = local_time->tm_hour;
    int min = local_time->tm_min;
    int sec = local_time->tm_sec;
    // printf("今天是 %d月%d日%d时%d分%d秒\n", month, day, hour, min, sec);

    lv_label_set_text_fmt(ui_date2, "%d-%d", month, day);

}

```

AI界面

长按文本框按钮进入弹出输入框与虚拟键盘，开始输入文本，只能输入英文文本内容，使用函数获取完整的文本句子。输入完毕之后点击文本框按钮提交输入内容。

```

void ui_event_inputlogo(lv_event_t * e)
{
    static bool session_initialized = false;

    lv_event_code_t event_code = lv_event_get_code(e);
    lv_obj_t * target = lv_event_get_target(e);

    // 创建会话 - 修复API密钥传递问题 使用硬编码API Key
    // 只在首次调用时创建session
    if (!session_initialized) {
        session = deepseek_create_session("----api-key----");
        if (!session) {
            fprintf(stderr, "创建会话失败\n");
            return;
        }
        session_initialized = true;
    }
}

```

```

    if(event_code == LV_EVENT_LONG_PRESSED )
    {
        lv_obj_clear_flag(ui_TextArea1, LV_OBJ_FLAG_HIDDEN);
        lv_obj_clear_flag(ui_Keyboard1, LV_OBJ_FLAG_HIDDEN);
        lv_obj_set_pos(ui_inputlogo, 117, 11);

        depmainlong(e);
    }
    if( event_code == LV_EVENT_CLICKED )
    {
        printf("inputlogo被长按!\n");
        depmaintalk(e);
        printf("退出eventcode! \n");
    }
}

```

```

#ifndef _DEPSKMAINCOPY_H_
#define _DEPSKMAINCOPY_H_

#include "lvgl/lvgl.h"

// ===== 数据结构定义 =====
// 存储HTTP响应
struct MemoryStruct {
    char *memory;
    size_t size;
};

// 存储API响应内容
struct APIResponse {
    char *content;
    int prompt_tokens;
    int completion_tokens;
    int total_tokens;
};

// 会话上下文
typedef struct {
    char *api_key;
    int total_prompt_tokens;
    int total_completion_tokens;
    int total_all_tokens;
} DeepSeekSession;

// ===== 封装函数声明 =====
/*****
 * @brief libcurl回调函数
 * @param contents
 * @param size
 * @param nmemb
 * @param userp
 * @return size_t

```

```

*****/
static size_t WriteCallback(void *contents, size_t size, size_t nmemb, void
*userp) ;

/*****
 * @brief Get the api key object
 * @return char*
 *****/
char* get_api_key();

/*****
 * @brief 调用DeepSeek API
 * @param api_key
 * @param prompt
 * @return struct APIResponse
 *****/
struct APIResponse call_deepseek(const char *api_key, const char *prompt) ;

/*****
 * @brief // 显示令牌使用信息
 * @param prompt_tokens
 * @param completion_tokens
 * @param total_tokens
 *****/
void print_token_usage(int prompt_tokens, int completion_tokens, int
total_tokens) ;

/*****
 * @brief 创建新会话
 * @param api_key
 * @return DeepSeekSession*
 *****/
DeepSeekSession* deepseek_create_session(const char *api_key);

/*****
 * @brief 销毁会话
 * @param session
 *****/
void deepseek_destroy_session(DeepSeekSession *session);

/*****
 * @brief 发送消息并获取回复
 * @param session
 * @param message
 * @param prompt_tokens
 * @param completion_tokens
 * @param total_tokens
 * @return char*
 *****/
char* deepseek_send_message(
    DeepSeekSession *session,
    const char *message,
    int *prompt_tokens, // 返回本次提示令牌
    int *completion_tokens, // 返回本次完成令牌
    int *total_tokens // 返回本次总令牌
);

```



```

/*****
 * @brief 获取会话令牌统计
 * @param session
 * @param total_prompt
 * @param total_completion
 * @param total_all
 *****/
void deepseek_get_token_stats(
    DeepSeekSession *session,
    int *total_prompt,
    int *total_completion,
    int *total_all
);

/*****
 * @brief Get the full text input object
 * @param dest
 * @param dest_size
 * @note 使用lv_textarea_get_text可能无法获取全部文本内容，只能获取单个单词
 *****/
void get_full_text_input(char *dest, size_t dest_size) ;

/*****
 * @brief 创建会话
 * @param e
 * @details 加入LVGL，在函数void ui_event_inputlogo(lv_event_t * e)
 *           中初始化，长按召出键盘与输入框，点击提交输入内容。
 *           但是将创建会话的功能剥离出去，该函数已经可有可无了
 * @return int
 *****/
int depmainlong(lv_event_t * e) ;

/*****
 * @brief 封装会话函数
 * @param e
 * @return int
 *****/
int depmaintalk(lv_event_t * e);

#endif

```

4. 快速开始

4.1 环境要求

技术栈说明

- 操作系统

- windows 10/11
- Ubuntu 20.04

- 开发环境：

- IDE：VSCODE
- 编译器：gcc、arm-linux-gcc
- 构建工具：CMake

- 核心依赖库

- libcurl
- cJSON
- SDL2
- lvgl

- 图形界面

- LVGL 8.3
- Square Line
- SDL2

- API集成

- Deepseek

- 版本管理

- Github

4.2 安装步骤

克隆项目

[git@github.com](https://github.com):Atlas-F/AISmart.git

4.3 配置说明

库配置

openssl（使用1.1.1版本）（1系列版本使用config命令，其他不变）

源码下载：<https://openssl-library.org/source/>

解压：tar 命令

配置并编译：

./configure --prefix=/opt/openssl/install-openssl no-asm（no-asm 不使用汇编进行加速）

make CC=arm-linux-gcc

(如果出现'-m64'的报错, 进入Makefile中删除所有的'-m64'即可, (因为我们的开发板是32位的))

make install

cURL (使用7.3.系列版本)

(curl需要配置编译)

源码下载地址: <https://curl.haxx.se/download/>

解压: tar -xvf curl-7.63.0.tar.xz

配置并编译:

Linux版本: (--prefix指定编译输出安装路径 --enable-static 指定编译静态库)

```
\    # ./configure --prefix=/opt/curl/install-x86 --enable-static
\    # make
\    # make install
```

ARM版本:(--prefix指定编译输出安装路径 --enable-static 指定编译静态库 CC指定交叉编译工具链, --with-ssl 指定链接openssl库 (支持https协议))

```
\    # ./configure --host=arm-linux CC=arm-linux-gcc --prefix=/opt/curl/install-libcurl --
with-ssl=/opt/openssl/install-openssl
\    # make
\    # make install
```

curl --version (查看版本信息, 并查看支持的协议)

cJSON

源码下载: 官网<https://sourceforge.net/projects/cjson/>

github<https://github.com/DaveGamble/cJSON>

(只用到cJSON.h和cJSON.c 两个文件)

4.4 运行示例

启动项目

./lvgl_sdl (ubuntu模拟器)

或者

./lvgl_fb (开发板)

5. 项目结构

AI_Smart/UBSDL-lvgl/

(AI_Smart/Armfb-lvgl/ 版本下有cjson.c和cjson.h)

└─ AI_Smart / # 使用SquareLine 快速设计导出的UI源码

```
├── depskmainCopy.c    # deepseek 接口文件
├── depskmainCopy.h    # deepseek 接口文件头文件
├── ...
├── CMakeLists.txt    # CMakefile文件
├── ui.c              # 事件与函数
├── ui.h              # 变量
├── ui_events.c       # 自定义回调函数与自定义函数
├── ui_events.h
├── lvgl /           # lvgl库
├── lvgl_drivers /   # lvgl库
├── /               # 文档
├── /               # 测试文件
├── /               # 示例代码
├── build            # 构建目录
│   ├── SimuTalk.txt # 对话模拟文件
│   └── ...
├── lvgl.h           # 配置文件
├── lvgl_conf.h       # 配置文件
├── lvgl_drv_conf.h   # 驱动配置文件
├── CMakeListsa.txt   # CMake构建主文件
├── main.c           # 主程序
└── README.md        # 说明文档
```

6. API 文档

- 接口说明
 - depskmainCopy.c # deepseek 接口文件
- 请求/响应示例
- 错误码说明

7. 开发指南

- 代码规范
 - UI相关部分遵循LVGL命名约定
 - 所有公共函数必须有Doxygen注释
 - 添加新功能时提供单元测试
- 提交规范
 - 新建分支 new feature

- git commit -m '时间戳-拉取分支-更新版本号-更新内容概述'
- 分支管理
 - 不要直接修改main分支
- 测试要求

8. 贡献指南

- 如何贡献代码
 1. Fork项目仓库
 2. 创建特性分支: `git checkout -b feature/new-feature`
 3. 提交更改: `git commit -m 'Add new feature'`
 4. 推送到分支: `git push origin feature/new-feature`
 5. 创建Pull Request
 - 欢迎通过GitHub提交贡献!
- PR 流程
- 开发环境搭建
- 联系方式

9. 常见问题

- FAQ
- 故障排除
 - 点击对话框刷新文本内容
 - 对话循环, 正常对话即可, inputlogo回调函数内初始化创建会话, 长按唤出键盘, 点击提交输入内容
 - 使用Square Line进行快捷设计导出时部分源码文件会被覆盖掉, 高危操作, 要注意提前保护
 - 在Ulevent.c中存储自定义的事件函数 (应该在外部再设置自定义文件将自定义函数存储在里面, 放置被覆盖)
自定义的函数不会被覆盖
- 已知问题
 - 开发板时间获取与实际不同, 需要手动校准
 - 对话的循环逻辑是死的, 只能强制退出
 - ARM架构找不到库文件, 已经发送到开发板上了, 但是开发板找不到 (已解决)
 - (高危: api key暴露, 不安全, 要使用更加安全的方式!)
 - 链接openssl配置支持https协议
 - 会话时可能出现段错误
 - 网络超时, timeout
 - 配置环境ssl证书

10. 更新日志

- 版本历史

- **2025.6.13 V1.0.1**
 - 休眠界面表情图间隔一秒随机切换
 - **2025.6.14 V1.0.2**
 - 手势滑动切换屏幕，左滑或者右滑
 - 向下滑动切换时间显示
 - 在第二屏设置模拟AI与人类对话，显示文本，使用文本文档模拟对话内容
 - **2025.6.15 V1.0.3**
 - 点击humanpanel，更新文本内容，humanpanel和AIPanel轮流显示对话文本，模拟对话
 - ubuntu可以正常模拟，开发板不行
 - **2025.6.16 V1.0.4**
 - 在ubuntu上面可以实现调用Deepseek，进行英文文本格式的输入与输出对话，但是逻辑顺序需要修正，循环的逻辑无法人为控制，应该在输出之后等待输入完成在进行下一次分析与输出
 - **2025.6.17 V1.0.5**
 - 修正了deepseek对话逻辑，获取完整输入，在inputlogo按键回调函数内部即创建会话，长按唤出键盘与输入框，点击提交输入内容。通过全局变量沟通LVGL和deepseek会话
 - 使用辅助函数获取完整文本框内容
 - **2025.6.20 V1.0.6**
 - 开发板上安装openssl链接curl库，否则不支持https协议，curl库与openssl配置版本匹配要求较高，先配置交叉编译openssl库（三步走：./configure -- make -- make install），再配置curl库。
 - SSL证书问题待解决
- 重要更新
 - 破坏性变更

11. 版权和许可

- 开源协议
- 作者信息：LFG
- 相关链接